Initially q_1, r_1, s_1 and all p_i are spawned. Also, $\mathtt{x}_i = \mathtt{y}_i = \mathtt{z}_i = 0$, for all i.

p_i :	q_i :	r_i :	s_i :
if $\mathtt{x}_i$ = 1 && i < n then	$\mathtt{y}_i$:= 1;	m := $\mathtt{y}_i$;	n := $\mathtt{z}_i$;
spawn q_{i+1};		if m = 0 then	l := $\mathtt{y}_i$;
spawn r_{i+1};		$\mathtt{z}_i$:= 1;	if n = 1 then
spawn s_{i+1};			if l = 0 then
			$\mathtt{x}_i$:= 1;

Fig. 10. The program of Fig. 2 extended with more processes and global variables.

- Claim 7.6 is replaced by $E'_q \vdash q.w'_q.w_q$;
- Claim 7.7 is replaced by $WI_{[E'_q.q]}(w'_q.w_q) \cap Sleep' = \emptyset$;
- in Claim 7.8, w_R is replaced by w_q.

In the text between Claim 7.8 and Claim 7.9, the text about race detection should be replaced by the observation that by choosing u as the subsequence of w with $w'_q.w_q \simeq E'_q u$, then the test at line 12 will succeed. The rest of the proof proceeds as for Algorithm 2.

9 COMPARISON AND TRADE-OFFS BETWEEN SOURCE-DPOR AND OPTIMAL-DPOR

In this section, we compare source-DPOR and optimal-DPOR on synthetic programs that highlight interesting performance differences, based on the number of explored interleavings, as well as in time and memory consumption. More specifically, we present:

(1) A program that shows that the use of wakeup trees allows optimal-DPOR to avoid exploration of an exponential number of sleep set blocked traces. In contrast, non-optimal DPOR algorithms in general, and source-DPOR in particular, are forced to explore these traces, and therefore can require arbitrarily more time than optimal-DPOR. As we describe, this does not happen only under the worst scheduling scenario, but also even when the scheduling algorithm that is employed is a "reasonable" one.
(2) A simpler program, which we also use as a benchmark in the next section, that also shows that a wakeup sequence is necessary for the optimal algorithm to avoid sleep set blocking.
(3) A program that shows that wakeup trees may make the optimal algorithm require exponentially more space than source-DPOR.

The first program, shown in Fig. 10, is an extended version of the example presented earlier in Fig. 2. It has been extended in the following way: n determines how many groups of four processes (which correspond to p, q, r and s from Fig. 2) are used. Each of the q_i, r_i and s_i processes operate on the respective $\mathtt{x}_i, \mathtt{y}_i$ and $\mathtt{z}_i$ variables. The code of the p_i processes reads $\mathtt{x}_i$ and only spawns q_{i+1}, r_{i+1} and s_{i+1} if the write on $\mathtt{x}_i$ by s_i has been completed. This pair of read-write operations is in a race. The other processes also have racing read-write pairs on variables $\mathtt{y}_i$ and $\mathtt{z}_i$.

When starting an interleaving to reverse the race of the top s_n process with the top p_n process, the only way for p_n to be removed from the sleep set is for each "lower" group to be scheduled exactly as shown in Fig. 3, as this is the only way for s_n to execute its write to $\mathtt{x}_n$. If a "wrong step" is taken at any point, s_n will not do the write and the respective trace will be sleep set blocked.

For $n = 1$ there exist four interleavings where the read on $\mathtt{x}_1$ by p_1 happens before the respective write by s_1 and one more where it appears afterwards, just as shown in Fig. 3. However with just the first step (which must be taken by r_1) specified, it can be the case for source-DPOR to schedule